

Lec 21 Introduction to VAE and the working of the Encoder

Lec 21 Introduction to VAE and the working of the Encoder

Lecture	21 / 63 + intro
Duration	41:14
Captions	English (auto-generated)
Words	6,035
Video	hOu8AoG83H4
Playlist	Fundamentals of Generative AI and LLMs

Auto-generated YouTube captions, lightly cleaned (removed [music] tags, grouped into paragraphs). These are **verbatim lecture captions**, not edited notes.

Transcript

[00:00] Hello everyone, welcome to the lecture series on fundamentals of generative VI and large language models. We are in the week three and week three is variational autoencoders. In the uh previous uh week we have seen uh autoencoders and in this particular session we'll be seeing the variational autoencoders and uh in today's session we will look into the working mechanism of encoder in VAE. Now uh from our previous session uh we have uh from our previous week autoencoders we have learned that autoencoders are mainly responsible for uh reconstruction. Then what is the motivation for variational autoencoders? The motivation here is generative modeling which means autoencoders is responsible for uh

[00:55] reconstruction and variational autoencoder uh uh encoders is introduced because now we want the generation we just want don't want a reconstruction we want a generation generating a new sample. So this is what is the overall objective. Reconstruction is very good but u it is just not a reconstruction. We want the new samples uh to be generated. So that is the reason why VAE is being uh introduced. So this is what is the overall motivation for VAE. [snorts] Now let us look into this particular uh uh example.

[01:35] Now uh here it is an iris data set and um a popular iris data set which is having n number of samples. Now let us see we have a data set which consist of n number of samples. How do you denote n number of samples? So here the first sample is x_1 x_2 and so on up to n th sample. Now here since it is an Iris data set there are three classes C_1 , C_2 and C_3 . So there are some samples which belongs to C_1 . There are some samples which belongs to C_2 and there are some samples which belongs to the class C_3 .

[02:19] What are these C_1 C_2 C_3 ? It is Iris, Sautazo, Versicol and virginica. Right? So these are the different classes. Now look at this diagram what is being given here. So all these are the samples which belongs to sorta. All these are the samples which belongs to one class which is virginica. Now when you look into the samples which are belonging to a particular class are all having the um features which are similar to each other. Now if you look into all of these samples the petal width value is very small right? If you look into the petal length the value is in this range. So all the samples belonging to a same uh class will be having a similar features.

[03:13] So can we say uh each class follow its own class conditional data distribution. What do you mean by data distribution? Here the values will be similar. The feature values will be similar to each other. So uh all the um samples which belongs to a same class follow a similar pattern. So you can see all the features which are there in this particular uh class follow have one type of a feature pattern and all the features which are there here follow one type of follow another type of a feature pattern right now what is uh VAE is an unsupervised uh model okay first one and it is used for generation right generating a new sample Now since it is an unsupervised model for sure during our training uh when the

[04:12] model is getting trained we will not be using any of the class labels. So this is what has been given here. Since it is an unsupervised learning model the class labels will not be used during the training process. If the class labels are not being used is it not the model is forced to learn the underlying pattern of the data. If how this pattern of this data is if the model can capture that it can generate a new sample.

[04:40] Right? So the the overall uh aim here is it has to learn the underlying pattern of the data so that it can generate a new data sample. Now there is one problem. What is the problem? The problem here is we do not know what is the true distribution of the uh given data set. So we doesn't know the true data distribution of the given data set. We doesn't know. Second, we even doesn't know the true mathematical um function.

[05:13] Uh you know the exact functional form also we doesn't know. What do you mean by functional form? Whether the original data is coming from a gaussian distribution or from a barnoli or a poisson distribution. We doesn't know anything about the original data set. All what we have is we just have some finite set of samples. And

what are these samples? These are our training samples. All what we have is the finite set of the samples which are drawn from this unknown distribution. So what is the um aim of a generative uh model? the um because VAE is a generative model, right? So what is the ultimate goal or aim of a generative model? It has to generate a data. If it has to generate a data, first it has to learn the

[06:07] underlying distribution. Right? So the first is it has to learn the underlying data distribution. Right? So once the underlying data distribution is learned then we can generate a new sample X new that will look exactly similar to that of the training data right that's our aim but what we doesn't know we doesn't know the original data distribution we doesn't know the original uh underlying data distribution but what we have we have the training samples with us few set of training samples and those samples actually belongs to this unknown distribution that is those samples belongs to this distribution only right so from these samples I cannot learn the exact why because I doesn't know I have

[06:58] only few set of samples from these few set of samples can I learn the approximate distribution possible right from this definitely we can learn the approximate distribution p_{θ} of x right So this approximate distribution whatever the model is learning it should be closer to the original distribution. So this is our ultimate aim learn the distribution with the few set of training samples what we have but ultimately reach to the original distribution.

[07:32] Right now what is this p_{θ} of x ? Is it not the distribution which is learned by the neural network? Um and how is that the model will be learning using some parameters θ and those parameters are weights and bias right now. So um with few set of training samples we are aiming to learn what is the underlying uh true distribution but with only few set we can learn the approximate distribution but not the exact. But at the beginning of the training definitely this distribution and this distribution will be very much different. But as the training is going on definitely we are going to update the parameters. When the parameters θ gets updated definitely this P_{θ} of X and the original distribution become

[08:30] closer. So over a period of time is it not the model has learned the underlying distribution. How it has learned the underlying distribution? What is that the model is having? The model is having the original few data samples which are drawn from those uh uh distribution from this distribution only original training samples. So though from those samples the model has learned this uh distribution. Initially the um learned distribution and the original distribution will be very far from each other. But over a period of time as the training is going on the uh model gets updated the model parameters gets updated with the uh updated parameters we can able to say that the learned distribution and the original

[09:19] distribution are closer. So now once the learned distribution is very much

closer to the original distribution is it not uh we can now sample uh we can generate a new sample from this uh distribution. Can I uh uh pick one value? Um so can we sample one value that sampled value will be closer to the original uh input. This is the overall objective or the overall working mechanism. However, we'll be going in detailed into the encoder part.

[09:56] So, what is that our ultimate aim from this particular slide? What is that you have to understand? We have a few training samples with us. From that uh few training samples, we should understand or we should learn or the model should learn the underlying data distribution. So, since you have only few training samples, you will be learning the approximate uh uh distribution. So uh that approximate distribution over a period of time as the training is progressing you will be able to reach to the original distribution by updating the parameters.

[10:33] With this let us start uh uh the encoder part. So uh whoever whenever you are listening uh to this particular uh session I request the uh learners to read uh or understand encoder and decoder together only then you'll be understanding the overall working mechanism. So in this particular session intentionally we are going very slow so that you will understand every part of VAE. Uh in this session I'll be covering encoders. In the next session, we'll cover the decoder. Towards the end of the next session, you will get the complete clarity of the overall working mechanism of VAE. If there are some parts if it is not clear to you, please wait until you complete the next session, you'll be able to understand

[11:20] that. Okay. Now, uh how uh let's go back and then understand how uh autoencoders have worked. We had an input. That input was passed to an encoder. Towards the end of the encoder we were getting the latent uh space um or latent representation. This latent representation is the um capturing the important variations or the most important um representation of the original uh input uh in a uh much compact representation. This is passed to a decoder right and uh towards the end of the decoder we are going to get the original uh uh data or we are going to get the reconstructed one. That reconstructed one you're going to compare it with the um original input. If the difference is too much you'll back propagate you'll

[12:11] update the parameters and over a period of time once the parameters are optimized you are going to get the reconstructed output which is as close as the original input. This is what is uh the uh a autoencoders. But what we are discussing now is a variational autoencoder. Uh if you if the working mechanism remains the same then um the whole intention of generating the data is lost. Right? So definitely there is some variation here. What is that variation? Let us understand step by step slowly. Here the first is there are two distributions. The first distribution here is called data space distribution. What is data space distribution? We have uh the data set original data set what we have with us.

[13:02] All that data set belongs to some distribution. That distribution we call it as a p data of x . So what is this? It is the distribution of the samples x_1 to you know x_n . Here we have considered a few samples. It is x_1 to x_n . Right? So all these are the um uh the data distribution of the original data set. The true distribution of the original data set. The second uh distribution what we have is a latent space distribution and that latent space distribution we we denote it as p of z . z is a latent space representation and the distribution of this is P of z . So we have two distributions P data of X and P of z .

[13:51] What is P data of X is the original samples distribution and z is a latent representation. That distribution is P of z . Okay. Now if you look into these two X and z lives in two different spaces. What do you mean by they live in two different space? The dimension and the structure of X is different and the dimension of z and its structure is different. Right? So because uh X lives in an input uh space and z leaves in an latent space, we need some probabilistic way of mapping these two.

[14:33] So this is what is given. Since x and z leave in two different spaces, the meaning x and z has um different dimensionality and then the structure. Therefore, the input x belongs to a data space and the latent variable z belongs to a latent space. Because they live in different space, we require some way of mapping these two. What is that way of mapping? It is an probabilistic uh mapping. Therefore in autoencoders the first component was encoder. Second component is latent space. Third component is decoder. But in VAE we have an probabilistic encoder and probabilistic decoder. Right? That's the um first difference that is encoder but here it is an probabilistic encoder.

[15:29] There we had a decoder but here it is an probabilistic decoder. So we need an probabilistic way of mapping from X to Z z to X dash or X hat. So the very first probabilistic mapping we call it as an approximate priority distribution. At an encoder site the role of an or the goal of an encoder is to learn the approximate priority distribution. Okay. So here is the first term uh where you need to understand uh encoder has to learn what encoder has to learn. Encoder has to learn something and then it has to give us the output. So and there in autoencoder whatever is the output of an encoder we call it as an latent space. So similar way here also encoder has to learn something. What is that the

[16:21] encoder has to learn? The encoder has to learn the true posterior. Okay. So what is the true posterior? What is the probability that we get z given the input x ? That is the true posterior. The true posterior is p of z given x . What is this? What is the probability that you get the latent space z given the input x . But encoder will not directly learn this true posterior. Which means encoder will not learn this. Why it will not learn the true posterior? Because true posterior calculation is very difficult. Why true posterior calculation is difficult? We'll see that shortly. But for now you understand that uh encoder will not directly learn the true posterior. Okay.

If encoder does not learn the true posterior, encoder

[17:20] has to learn something. What is that encoder is learning? It is learning the approximate posterior. What is approximate to posterior? Approximate to posterior is denoted in this way. Ultimately what is the probability that you will get the z given x that is ultimate one. But instead of learning p of z given x true posterior the uh encoder is actually learning the approximate posterior. What is approximate posterior? It is Q of z given X by some learnable parameters that can be weights by some learnable parameters. Right? That is what so what is the um what is this particular term is saying what is the probability of the latent variable Z given the X given the data sample X that is what is the role

[18:16] of an encoder. The role of an encoder has to learn something that something here is a posterior uh and it will not learn the true posterior it is learning the approximate posterior. So second one is at the decoder side. What is a decoder? What is the probability of getting uh this $\hat{x}$ given the latent variable z ? So uh this is what is the um you know decoder role. What is the probability that the decoder can generate the data X given the latent variable z ? So is it not z is passed as an input to a decoder. So ultimately what is the probability that you get this output when you pass this as an input that is what is the decoder role and decoder um at the decoder side we call it as a likelihood distribution. At

[19:14] an encoder side we call both has to learn the distribution because it is a probabilistic encoder. Probabilistic encoder has to learn the distribution. What distribution the uh encoder is learning? What is the probability of a latent variable z given x ? What is the uh decoder here? It is a probabilistic decoder which means it has to learn some distribution. What distribution it is learning? what is the probability that you get the um x given the latent variable z . Okay. So now let us uh understand the encoder part. What is the ultimate aim of an encoder? Encoder has to learn this. But u encoder is learning the approximate. Why encoder is learning the approximate? Why not true? You will get some portion of an answer here.

[20:04] Okay. So the primary objective of an encoder is to learn this true posterior. What is posterior here? The distribution of a latent variable z given an input x . So it has to learn this p of z given x . But what is how is that you calculate p of z given x from the base theorem? Is it not p of z given x is uh represented in this way using a base rule. In this what is this particular term? This is called true posterior distribution. This is this P of z we call it as a prior distribution. And what is this P of X given z ? This is called a likelihood distribution and denominator P of X is an evidence. So this is all each of these terms. This is from our fundamental ML we know this. So from the base rule P of Z given X is

[21:03] denoted in this way. Now our ultimate aim of an encoder is to learn this P of Z given X . But we said that it is difficult. Why it is difficult? Because of this difficulty

instead of learning P of Z given X encoder is learning the approximate distribution or approximate posterior. Now uh why this p of z given x is difficult to compute is look at the denominator term p of x . What is p of x ? How is that we calculate p of x ?

[21:37] p of x computation is very hard here. Why uh it is hard? Because it requires the uh integration over all the possible values of a latent variable z . So this one statement you hold on. We will come back to this statement in a while. Uh if not in this particular session when you come to an decoder you'll be able to understand this. So uh true posterior is p of z given x that can be calculated using base rule. But the base rule denominator is difficult to compute. Why it is difficult to compute? because it requires integration over all the possible values of a latent variable z . This is difficult part. Let's put put this on hold. Why it is difficult?

[22:26] We'll come back because of this instead of um VAE instead of directly computing the true posterior it is using an approximate. From this slide you should only understand one point and that one point is instead of going for true posterior we are going for an approximate. Why we are going for approximate? Because true posterior denominator term is difficult to compute. Why it is difficult to compute? Because it requires an integration over all the possible latent variable z . So this is uh something which is still on hold. We'll come back to this particular uh question.

[23:10] in a while. Right? So now let us move to what we have understood is okay encoder has to learn the approximate posterior. Now let us look into this particular uh z . Now in VAE z is not a single value. Why it is not a single value? Because ultimately from 100 you cannot bring down to one feature representation. We cannot right which means most of the information you have compromised there. So it cannot capture the 100 features into one. No. So it can never be an single uh uh number. It has to be an vector. Okay. If you consider one example, if there are 100 features, these 100 features when you pass it to an encoder, you get the output 25. If a latent space size is 25 which means 100 neurons 100 feature values have been

[24:12] compressed and represented as 25 important um representation. So we have not deleted 75 that 100 is being compressed to 25. So which means it is not a single number. It is having a 25 features that is what is a vector of a size 25. So what are we understanding here? VA is not a single number but it is an vector. Okay. And uh in this particular example here it is a vector of size 25. Okay. What is the uh requirement? Uh go back to autoencoders. Uh input when it is passed to an encoder. encoder has to uh take up the uh important representations and then it has to represent in a much compact lower dimensional latent space.

[25:04] Correct? So if that is the case, VAE requirement is also the same. The purpose of an encoder has to learn the meaningful latent representation, right? uh uh representation of what of the given input data uh in in a much uh you know lower dimensional latent space. If that is the case, I have an input. This input has to be

represented in a lower dimensional latent space. What is the distribution of that latent space? What is P of z here? Do we know? Answer is no. We doesn't know what is the structure of this latent space.

[25:39] Therefore the first assumption what we are doing ultimately encoder has to learn this distribution that distribution we still doesn't know and then now we are saying that p of z so again there is one more distribution here so we doesn't even know that therefore the first assumption that we are doing here is the prior distribution p of z is actually a gshian this is the first assumption that we are doing not the first it is an assumption that the prior distribution p of z is gshian. What are the two parameters of gshian? Is it not the mean and variance?

[26:18] So uh now um so this is what are the parameters and this is an prayer probability. So p of z is a prior distribution prior probability and we are assuming it to be a goshian. Now you can ask why goshian? There are so many other distributions. When there are so many distributions, why did we assumed it to be a gshian only? For now just understand because gshian is very simple and then mathematically it is convenient. Why mathematically it is convenient? Because it just have only two parameters mean and variance and uh and we'll come back to the second point. Sampling becomes easy. For now just remember theoretically only these two that is gshian is simple and mathematically convenient and sampling becomes easy

[27:07] right. So now uh before we come to an end of this encoder let us quickly understand what is an uh role of an encoder. Encoder has to learn something. What is that it has to learn? It has to learn the um approximate posterior. And what exactly is this? It has to learn the distribution over a latent variable. The that is uh it has to learn this um know what is the probability that you get the latent space z given the input x . This is what the uh goal of an uh encoder. Now we have also seen it cannot learn the true because denominator is difficult to compute. Why we have still not answered to it. Although we know that integration is integral over all the values but what is that we have still not understood. Okay. Now when

[28:02] this is the aim of an uh uh encoder uh which means it has to learn the distribution. If it has to learn the distribution which means some distribution it has to learn. Now instead of uh allowing this encoder to learn some complex distribution can we again give one more restriction here? Let the this distribution is once again a gshian. Okay. So instead of allowing this distribution to be any complex distribution, we assume it to be gshian which means this um has to give two outputs. What are those two outputs?

[28:40] Mean and variance. Right? Are you getting this? Encoder has to learn something. What is that encoder has to learn? Encoder has to learn mean and variance is what encoder has to learn. Why encoder has to learn mean and variance only? Because we have assumed it to be gshian. Why we has why we assumed it to be gshian?

Look at this. Now when input is passed to an um encoder ultimately we have to get the latent space. Right? Now already latent space we have assumed it to be a gshian.

[29:16] We have already assumed it to be a gshian from our previous slide. We have seen if this is a gshian from input only we get this latent space. So if that is the case so um encoder also is assumed to be a gshian. So this p of z which we have already assumed to be gshian serves as a reference um for encoder. So that encoder also takes it as an goshian. Now there are two things here. One is a prior distribution. P of z is also a gshian. Then q of z given x is also a goshian. So if you assume this as gshian, you can compare these two very easily. Now from a very long time we are using the word gshian right. Why?

[30:06] When we have so many other distributions, why gshian only? Because the gshian distribution can be represented just with the two parameters and those two parameters are mean and variance. So which means the encoder has to learn ultimately only two parameters that is mean and variance. Encoder has to output mean and variance which means training is easy because it has to learn only uh two parameters right. Second sampling becomes easy. What do you mean by sampling? Once z uh mean and variance once encoder outputs mean and variance ultimately we should get z z is obtained by sampling from this distribution. So once encoder uh encoder gives mean and variance we can sample z from this gshian distribution. So

[30:58] that is uh the um uh you know two important points why the model assumes a gshian but not any other distribution. Now here what has happened? Um uh we have an input. This input is passed to an encoder and this is an probabilistic encoder and we are getting an latent space. This is gshian. This is gshian. So both here what is this? This Q of z given X with some learnable parameters and P of z both are gshian here uh these two are gshian. Now how we are ensuring that both are gshians here both are following or um encoder probabilistic encoder is closer to the prior who is ensuring that both are following the gshian only it is we have an k divergence k divergence is a regularization term which we'll be adding to a loss

[32:06] function so uh and what is a loss function here definitely it has to be a reconstruction loss. So this KL divergence always ensures that encoder distribution remain closer to prior. Right? What is a prior gshian? What is uh encoder distribution? It assumed to be gshian but ultimately it has to adjust its parameters so that it remain closer to the prior distribution. So KL divergence takes the term. So this is all the theoretical concept until we see one mathematical example. Uh whatever we have seen theoretically may not be much clear to you. Now let us understand the entire working mechanism of an encoder with this one uh example.

[32:56] So so far if something is a little doubtful to you also you'll be able to understand that here. Now we have an input. this input has to be passed to an neural

network. Uh so here this probabilistic encoder is a neural network. So when you say neural network you you imagine that neural network as an multi-layer perceptron. If it if it is a multi-layer perceptron you'll have any number of hidden layers. Every neuron is connected to every other neuron. You have weights, you have bias, you have an activation function.

[33:28] Although we will be seeing one detailed numerical example in this week but uh to understand uh uh let me explain uh uh theoretically here you have hidden layers uh connections um weights bias then we have an activation function all of this is fine can you imagine uh what will be the end of an um the last hidden layer you'll get some values right so let us hold stop there and let us come back to this example you have an input input is passed to an um encoder. Here encoder is a simple neural network. If it is an uh tabulated data, use a multi-layer perceptron. If input is an image, use an CNN based neural network.

[34:13] Now when it is passed towards the end of an last layer, some output we should get. Now let us see what is that output. that output here is mean and then the log variance. What we get towards the end of an encoder is you remember uh uh the role of an encoder is to learn uh this uh q of z given x which is nothing but an um you know approximate posterior distribution. So it has to give us some output. What output? You have assumed this to be a gshian. If you have assumed this to be a gshian, gshian will be having only two outputs that is mean and variance. But what output you have got here is log variance. So since gshian requires a mean and variance.

[35:03] This log variance you convert that into a variance. How will you convert a log variance into a variance? A simple math you know substitute in this formula and convert log variance into a variance. Okay. Now the most important this how what should be the latent space size it's an hyperparameter that means in the code you'll be writing what should be the size of your uh size of the latent space here the size of the latent space what we have given is five so which means we are going to get z_1 z_2 z_3 z_4 and five which means we are going to have z_1 z_2 z_3 z_4 and z_5 this will be what we are going to obtain at the latent uh space. Um this is what is the output. Now uh what is the output that we got from uh uh

[35:58] encoder because it is a gshian it has output it has given us mean and variance. How many such mean how many such variance that depends on the latent space size. Since the space size is five, we are getting five such values and five such uh these values. Okay. [snorts] Now once we get this, you remember z has to be sampled from this distribution, right? What is the meaning of this? Now this is one distribution. Why? You have taken this mean and this log variance later you have converted into variance. So from this mean and variance sample one z . So what do you mean by sampling? Randomly picking one value. Okay. So when you pick randomly one value is it not that value is this right? Similar way from this particular

[36:57] mean and variance from uh this and this from mean and variance uh pick randomly one number that becomes zed 2. Similar way from this mean and variance uh sample one point that is zed3. Similar way zed 4 and zed 5. What did you pass? You passed one input that for that input you got mean and variance and then you have sampled zed from this distribution. Sampling zed from this distribution is randomly picking one value from this uh distribution. Now we got these five.

[37:36] This five can we pass it to a decoder? We have still not seen what is the decoder. Let us uh not bother about it. So now we got this as an input that is five values. These five values are passed to a decoder. Now we'll understand decoder in a while. So this is what we have obtained five uh such uh values five this and five these values. Let us take one example and then understand here mean are five and the log variance are five and you have converted log variance to variance and finally this is the variance. This is the variance and this is the mean. Okay.

[38:17] Now, so this is what we have obtained these two. What is that we have to now do? We have to sample zed from this distribution. So, uh how zed is sampled? Zed is sampled from this distribution. This is the mean. This is the variance. This is the mean. This is the variance. So, what we have? we have zed 1, zed 1, zed 2, zed3, zed 4 and zed 5. So we totally get five values. So when we randomly now sample one value from each gshian distribution. So from this can I pick one value? Can we pick uh randomly when we pick one value? Is it not uh zed 1 is 0.65?

[39:04] Why 0.65? It is random. It can be any number uh uh closer to this particular mean. So 65 then zed 2 from this distribution when you randomly picked one value we got this similar way zed3 zed 4 and zed 5. So look at this finally the sampled latent vector what we have obtained is this. Now is it the only one value that we are getting? uh uh now randomly when you pick one more value from this particular distribution it can be even 0.45 also right it can be even 0.55 also it can be even 0.67 6 7 how many such values are possible? infinite such values are possible right so I'll stop at this point of time I gave one hint to you that is infinite such values are possible so this is one uh uh reason

[40:02] why the denominator is difficult okay so uh what is that we have understood so far input has to be passed to an encoder and encoder has to output us something and what is that encoder is giving us the output here it is mean and variance. Why mean and variance only? Because it is assumed to be gshian. Why it is assumed to be gshian? Because latent space p of zed we have assumed to be gshian. So uh this is the overall working mechanism of an uh encoder. In the uh next session we will be looking into the decoder part of the VAE and then towards the end of that what all questions we have why denominator is difficult then uh how VAE is called a generative model why not it is called a reconstruction reconstructive all that

[40:58] we will be able to answer in the uh next session where we'll be looking into

a decoder part and we'll also look into the loss uh function of and the VA. Thank you.